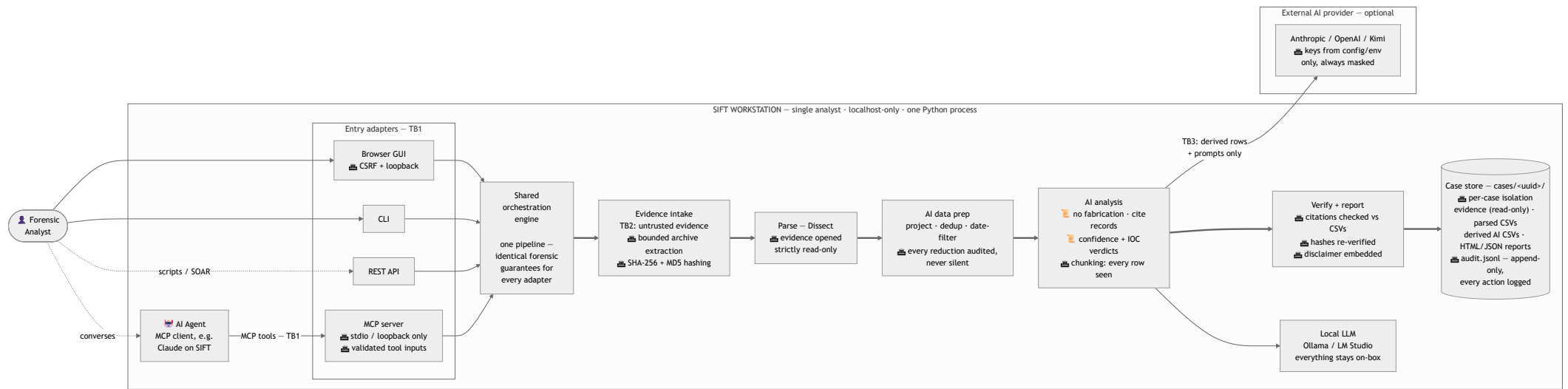


AIFT — Architecture Diagram

AI Forensic Triage (AIFT) is a locally-hosted forensic triage agent running on the SANS SIFT Workstation: evidence is parsed read-only with the Dissect framework, an LLM produces cited and machine-verified findings, and a self-contained HTML/JSON report is generated — every step captured in an append-only audit trail. An MCP server lets an AI agent on the SIFT box drive the whole workflow as tools.

1. System at a Glance

Legend — **TB#** = trust boundary · 🏠 = **architectural guardrail** (enforced in code; model output cannot bypass it) · 📄 = **prompt-based guardrail** (instruction to the model; verified downstream where possible)



TB3 is the only boundary where data can leave the host — and it is optional: the local-LLM provider keeps the entire triage on the SIFT workstation.

2. Architectural Pattern

Ports-and-Adapters (Hexagonal) core + Pipes-and-Filters pipeline, local-first single process.

- **Hexagonal:** GUI, CLI, REST and MCP are thin adapters over one shared orchestration engine — no adapter has its own pipeline logic, so read-only evidence, hashing and audit hold identically for agent-driven (MCP) runs.
- **Pipes-and-Filters:** the triage run is a linear chain of isolated stages, each reading and writing well-defined artifacts inside one isolated case directory.
- **Local-first:** one Python process; no database, no containers, no required cloud service.

3. Trust Boundaries

#	Boundary	Key enforcement (architectural)
TB1	Analyst / AI agent → AIFT	Loopback-only binding; CSRF on GUI mutations; MCP via stdio or localhost, remote needs explicit opt-in; typed + validated MCP tool inputs
TB2	Untrusted evidence → parser	Dissect opens evidence read-only, originals never written; bounded archive extraction with path-traversal and symlink rejection
TB3	AIFT → AI provider	Only projected/deduplicated derived data crosses; API keys masked everywhere; fully avoidable via local LLM
TB4	AI output → forensic report	Model output is treated as unverified : citations spot-checked against source CSVs, hashes re-verified, reasoning text never exported, human-review disclaimer mandatory

4. Prompt-Based vs Architectural Guardrails

 Prompt-based (requested behavior)	 Architectural backstop (enforced in code)
Never fabricate records, timestamps, values	Citation validator spot-checks every cited row/timestamp/column against source CSVs
Cite specific records for every finding	A <code>row_ref</code> column is injected into the data, making citations machine-checkable
State uncertainty; rate confidence HIGH/MED/LOW	Findings ship with the mandatory human-verification disclaimer in every report
Classify each IOC: Observed / Not Observed / Not Assessable	IOCs are code-extracted from the analyst context and listed in the prompt
Note evidentiary absences instead of guessing	Chunked analysis guarantees the model actually saw every row — no sampling

Design stance: prompt guardrails are *requested*; the architecture *checks* what matters forensically. The citation validator exists precisely because the no-fabrication rule cannot be assumed.

Appendix A — Guardrail → Enforcement Map

Guardrail	Type	Enforced at
Evidence read-only; originals never modified		<code>app/parser/core.py</code> (Dissect <code>Target.open</code>), intake never writes to sources
SHA-256 + MD5 on intake, re-verified before every report (PASS/FAIL/SKIPPED)		<code>app/utils/hasher.py</code> , orchestrated by <code>app/automation/engine.py</code>
Append-only audit trail; closed action-type allowlist; fsync per write		<code>app/logging/audit.py</code> → <code>cases/<uuid>/audit.jsonl</code>
Per-case isolation; derived AI CSVs never replace parsed source CSVs		<code>app/logging/case_manager.py</code> , <code>app/analyzer/data_prep.py</code>
No silent data reduction — projection/dedup/date-filter all audited		<code>app/analyzer/data_prep.py</code>
AI citations spot-checked against source CSVs		<code>app/analyzer/citations.py</code>
Every row reaches the model (chunking + hierarchical merge, no sampling)		<code>app/analyzer/chunking.py</code> , <code>app/analyzer/chunk_merge.py</code>
Loopback-only GUI/REST; MCP stdio default, remote explicit opt-in		<code>aift.py</code> , <code>aift_mcp.py</code>
Bounded archive handling (zip-bomb / traversal / symlink defenses)		<code>app/evidence/archives.py</code>
API keys from config/env only; masked in all API responses; secret-free MCP output		<code>app/ai_providers/base.py</code> , <code>app/routes/state.py</code>
Model reasoning channel excluded from reports, exports, audit logs		<code>app/ai_providers/progress.py</code> , <code>reporter/exporters</code>
AI-assistance disclaimer hardcoded into report template		<code>templates/report_template.html</code>
Forensic persona: no fabrication, cite records, confidence ratings, state uncertainty		<code>prompts/system_prompt.md</code>
IOC verdict discipline; default DFIR checks (persistence, lateral movement, ...)		priority directives, <code>app/analyzer/ioc.py</code>
Per-artifact analysis guidance; note absences (cleared logs, gaps)		<code>prompts/artifact_instructions*/</code>
Final context-reminder block survives context-window truncation		<code>prompts/artifact_analysis.md</code>

Appendix B — Output Pipeline (data products, in order)

evidence (read-only)	— hashes → audit.jsonl
↳ parsed/*.csv	full Dissect output, never modified
↳ parsed_deduplicated/*.csv	derived AI input (projected/deduped/filtered, audited)
↳ analysis_results.json	cited AI findings (citation-validated)
↳ report.html / report.json	self-contained, with disclaimer, hash verification + full audit trail

Every stage appends to `cases/<uuid>/audit.jsonl` (append-only, UTC, fsync'd), so the chain of custody from raw evidence to AI finding to report is reconstructable after the fact.

Appendix C — SIFT Deployment Notes

- AIFT runs **on** the SIFT workstation as a single Python process (`python aift.py` , `aift_cli.py` , or `aift_mcp.py`) — no Docker, no services, no database.
- Evidence collected with standard SIFT/DFIR tooling (disk images, KAPE/ `acquire` triage packages, archives) is consumed in place, read-only; parsing uses the **Dissect** framework.
- With a local LLM endpoint (Ollama/LM Studio) the entire triage — including AI analysis — stays on the SIFT box; cloud providers (TB3) are an explicit opt-in.
- The MCP server turns SIFT into an agent-operable triage station: the agent gets validated tools, while every forensic guarantee is enforced by the shared engine — not by the agent's good behavior.